

Investigating the Effect of Multiple CPUs on Run Time Reduction of Over-sampling of Imbalanced Fraudulent Transaction Dataset

Abstract— This research investigates the effect of parallelization on the reduction of the run time of training a fraudulent transaction classification model. In the literature, the most time-consuming process, which is the over-sampling of the imbalanced fraudulent transaction dataset, is implemented through map reduce, so that the overall training time can be reduced through data parallelization. This research used Welch's t-test to verify that parallelization effectively reduces the training time by comparing the run time of single CPU and multiple CPUs. This research also conducted two experiments by manipulating the number of workers and the number of CPUs per worker respectively. Through the two experiments, this research found that multiple CPUs in the same node yields shorter training time compared to multiple CPUs in different nodes. This research used Welch's t-test to verify this observation by comparing the run time of 6 workers with 1 CPU each and 2 workers with 3 CPUs each.

Keywords— Docker, data parallelism, map reduce, multi-CPU.

I. INTRODUCTION

A. Background

Unauthorized or fraudulent credit card transactions is a serious crime that can cause severe loss in the card holder's money. Fraudulent transactions can be caused by compromised bank account due to hacking activity. To protect bank account holders from severe loss of money in the event of compromised account, the bank should implement a mechanism to detect fraudulent transactions and stop those suspected fraudulent transactions to be processed. Due to the number of transactions in a bank comes in high volume and high speed, it is not practical to detect fraudulent transactions manually. So, to detect fraudulent transactions automatically in real time, one possible solution is training a fraudulent transaction detector using machine learning based on actual transaction data. The work of [1] is a recent effort to produce a machine learning solution for detecting fraudulent credit card transactions.

One problem when training a fraudulent transaction classification model through machine learning is severely imbalanced dataset. This is the problem faced by the authors of [1] because the dataset that they used is highly skewed. In fact, the dataset only has 492 fraudulent transactions out of 284807 transactions, which is approximately 0.172% [1]. This will cause the model to bias towards the majority non-fraudulent class and has a weak performance on the minority fraudulent class.

The authors of [1] tackled this problem by oversampling the minority class, which is fraudulent transaction, using synthetic minority oversampling technique (SMOTE) and edited nearest neighbors (ENN). SMOTE produces synthetic instances of the minority class based on the value of the close neighbors, whereas ENN removes those instances which are misclassified with its three nearest neighbors [1]. Combining these two paradigms produces a clearly separated instance space with balanced class distribution, which solves the problem of imbalance class distribution.

SMOTE and ENN compares the distance between each instance to find the nearest neighbors of an instance. This requires large amount of computation in the case of large dataset. One way of implementing SMOTE and ENN more efficiently is by parallelizing the computation through distributed systems. This is exactly the way how the authors of [1] reduces the computation time for SMOTE and ENN. Figure 1 illustrates how the authors of [1] implement SMOTE and ENN through distributed system and how SMOTE and ENN fits in the overall pipeline of training the fraudulent credit card transaction classification model.

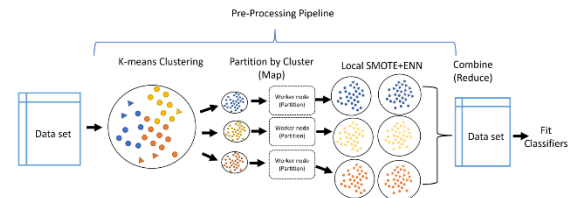


Figure 1 Overall pipeline of the work of [1]

As shown in Figure 1, the whole dataset is first clustered into several groups using k-means clustering and the data instances are distributed to each worker based on its cluster [1]. In the map phase, each worker implements SMOTE and ENN to the instances in the same cluster and produce an output which the key is "1" and each value is an array of all predictor attributes and the class label of one instance [1]. In the reduce phase, all instances are grouped in the same machine since all instances have the same key [1]. The output of the reduce phase is fed as the dataset to train a random forest model to detect fraudulent transaction [1]. Clustering has made SMOTE and ENN parallelizable because each worker can take one cluster. Data parallelism is implemented here because each worker executes the same SMOTE and ENN function on different subsets of the dataset.

B. Problem Statement

With a distributed SMOTE and ENN implementation through map reduce in the training pipeline of a fraudulent transaction classification model in the work of [1], the question that arose is whether parallelization is able to reduce the run time of the training process. To answer this question, this project compares the run time of running the training process implemented using the code from the work of [1] on 1 worker with 1 CPU and multiple workers with multiple CPUs.

If parallelization of SMOTE and ENN is able to reduce the run time, the second question arose is whether multiple CPUs in the same node or multiple CPU in different nodes yield a better performance. To answer this question, two experiments are conducted in this research. The first experiment is increasing number of nodes while fixing the number of CPUs of each node at 1 (multiple CPUs in different nodes), whereas the second experiment is increasing the number of CPUs in each node while fixing the number of nodes at 2 (multiple CPUs in the same node).

II. TOOLS, DATASET, AND SOURCE CODE

A. Tools and Platform Used

In this project, the source code written by the authors of [1] is used. There are three set of codes in the repository of [1], each representing one training approach. The code in the file named “Local MapReduce.ipynb” is used because it implements the approach of parallelizing SMOTE and ENN as it is discussed in section I.A. The authors of [1] set the number of clusters as 6 in this code. This configuration is left as it is.

The platforms used in this project to implement parallelization of SMOTE and ENN is Docker Compose. Docker Compose is used for implementing Docker container in an environment that mimics distributed multi-node system using a single machine.

B. Datasets Descriptions

The dataset used in this project originates from [2]. In fact, this is also the dataset used in the work of [1]. The dataset is the real transaction data of European credit card holders in September 2013 [2]. In this dataset, there are 284807 instances with 31 attributes [2]. Each instance represents one credit card transaction [2]. There are 492 fraudulent

transactions and 284315 non-fraudulent transactions in the dataset [2].

To avoid the leakage of confidential information contained in the transaction data, the authors of [2] transformed the original data using principal component analysis (PCA) into 28 anonymized numerical attributes. The attributes that were not transformed are time, amount and class [2]. Time is the number of seconds elapsed from the time of the first transaction, whereas amount is the amount of the transaction, whereas class is whether the transaction is fraudulent [2]. The class “1” represents “fraudulent”, whereas the class “0” represents “non-fraudulent” [2].

III. IMPLEMENTATION

The code is run using Visual Studio Code and Docker Desktop. The official Apache Spark image is used as the base for each node in this section. Apache Spark uses the exactly same abstraction as the original Hadoop Map Reduce, but is much faster than Hadoop because its underlying framework is more efficient.

In order to run Apache Spark instances in Docker containers, a “DockerFile” is written with the help of co-pilot so that when each instance is initiated, the system can install required Python libraries, set the working environment, download the source code and the dataset and specify the port of the master node.

In order to use Docker Compose to spins up the distributed system, a “docker-compose.yml” file is written with the help of co-pilot to specify the number of worker nodes (replicas) and the number of CPUs and memory allocated for the master node and each worker node. In this section, the memory allocated for each CPU is kept constant at 1024 MiB. The number of workers and the number of CPUs per worker is edited based on the experiment settings.

IV. EXPERIMENT SETTINGS AND RESULTS

In this research, two experiments are conducted. The first experiment is increasing the number of workers while keeping the number of CPUs per worker constant. The number of workers of 1, 2, 3, 4, 5 and 6 are experimented. The number of CPUs per worker is fixed as 1. Table 2 lists the results for the first experiment.

Table 1 Results of the first experiment

Number of workers	Run 1 (second)	Run 2 (second)	Average run time (second)
1	84	84	84.0
2	66	60	63.0
3	54	53	53.5
4	59	59	59.0
5	56	66	61.0
6	66	66	66.0

As shown in Table 2, the average run time is the shortest when the number of workers is 3. The run time decreases from 1 worker to 3 workers and increases from 3 workers to 6 workers. This is because the communication overhead increases when the number of nodes increases. For this task and this dataset specifically, the communication overhead outweighs the reduction of workload per node when the number of nodes is more than 3.

The second experiment is increasing the number of CPUs per node while keeping the number of nodes constant. The number of CPUs per node of 1, 2 and 3 are experimented. The number of nodes is fixed as 2. Table 3 lists the results for the second experiment.

Table 2 Results of the second experiment

Number of CPUs per worker	Run 1 (second)	Run 2 (second)	Average run time (second)
1	66	60	63.0
2	31	31	31.0
3	26	27	26.5

As shown in Table 3, the average run time decreases when the number of CPUs per worker increases and the shortest run time is achieved when the number of CPUs per worker is 3.

Comparing the 6 workers with 1 CPU each and 2 workers with 3 CPUs each, the run time of the latter (26.5 seconds) is 59.8% shorter than that of the former (66.0 seconds), despite both of them utilizes 6 CPUs. This is because the speed of reading and writing data for multiple CPUs in the same node is significantly faster than that of multiple CPUs in different nodes.

V. VERIFICATION THROUGH HYPOTHESIS TESTING

To verify the claim that parallel computation yields shorter run time than serial computation, the run time of 2 workers with 3 CPUs each is compared with the run time of 1 worker with 1 CPU. The setting of 2 workers with 3 CPUs each is chosen to represent the parallel

processing group because it has the shortest run time.

Besides that, to verify the observation that the run time is faster in multiple CPUs in the same node compared to multiple CPUs in different nodes, the run time of 2 workers with 3 CPUs each is compared with the run time of 6 workers with 1 CPU each.

Each of the three settings are run for 10 times. The exact figure of each run for 1 worker with 1 CPU, 6 workers with 1 CPU each and 2 workers with 3 CPUs each are recorded in Appendix A, B and C respectively. The mean and standard deviation of each setting is shown in Table 3.

Table 3 Mean and standard deviation of each setting

Setting	Run time (second)	
	Mean	Standard deviation
1 worker with 1 CPU	82.2	2.898
6 workers with 1 CPU each	66.6	1.897
2 workers with 3 CPUs each	28.3	1.059

To verify whether parallel computation yields shorter run time, the run time of 2 workers with 3 CPUs each is compared against the run time of 1 worker with 1 CPU. The null hypothesis, H_0 , states that the run time of 2 workers with 3 CPUs each is longer or equal to from the run time of 1 worker with 1 CPU. The alternative hypothesis, H_1 , states that the run time of 2 workers with 3 CPUs each is shorter than the run time of 1 worker with 1 CPU. The significance level is set as 0.05. With $\bar{x}_0 = 82.2$, $s_0 = 2.898$, $\bar{x}_1 = 28.3$, $s_1 = 1.059$, the null hypothesis, h_0 , and the alternative hypothesis, h_1 , are expressed in the equations below:

$$H_0: \mu \geq 82.2$$

$$H_1: \mu < 82.2$$

After testing the hypotheses with Welch's t-test, it is found that the p-value is 3.602×10^{-15} . Since the p-value is less than 0.05, the null hypothesis, h_0 , is rejected. So, it can be said that the run time of 2 workers with 3 CPUs each is shorter than the run time of 1 worker with 1 CPU. So, it is statistically proven that parallel computation yields a shorter run time than serial computation.

To verify whether multiple CPUs in the same node yields shorter run time than multiple CPUs in different node, the run time of 2 workers with 3 CPUs each is compared against the run time of 6 worker with 1 CPU each. The null hypothesis, H_0 , states that the run time of 2 workers with 3 CPUs each is longer or equal to from the run time of 6

workers with 1 CPU each. The alternative hypothesis, H_1 , states that the run time of 2 workers with 3 CPUs each is shorter than the run time of 6 workers with 1 CPU each. The significance level is set as 0.05. With $\bar{x}_0 = 66.6$, $s_0 = 1.897$, $\bar{x}_1 = 28.3$, $s_1 = 1.059$, the null hypothesis, h_0 , and the alternative hypothesis, h_1 , are expressed in the equations below:

$$\begin{aligned} H_0: \mu &\geq 66.6 \\ H_1: \mu &< 66.6 \end{aligned}$$

After testing the hypotheses with Welch's t-test, it is found that the p-value is 5.950×10^{-18} . Since the p-value is less than 0.05, the null hypothesis, h_0 , is rejected. So, it can be said that the run time of 2 workers with 3 CPUs each is shorter than the run time of 6 workers with 1 CPU each. So, it is statistically proven that multiple CPUs in the same node yields shorter run time than multiple CPUs in different nodes.

VI. CONCLUSION

In conclusion, by using the map reduce code written by the author of [1], this research empirically proved that parallel processing of the map reduce yields shorter run time compared to serial processing of map reduce. This shows that parallelization of the over-sampling step can effectively reduce the run time. This is because with parallelization, the workload is distributed across node, so the processing time is reduced significantly.

This research also empirically proved that multiple CPUs in the same node yields shorter run time compared to multiple CPUs in different nodes. This is because the overhead of communication across node is significantly higher than that of communication within node.

VI. REFERENCES

- [1] S. Jajoo, "Big-data-Credit-card-fraud-detection," Git Hub, 1 October 2024. [Online]. Available: <https://github.com/SMJajoo/Big-data-Credit-card-fraud-detection>. [Accessed 2 June 2026].

- [2] Machine Learning Group, Université Libre de Bruxelles, "Credit Card Fraud Detection," Kaggle, [Online]. Available: <https://www.kaggle.com/datasets/mlg-ulb/creditcardfraud>. [Accessed 2 June 2026]

Appendix A: Run time of 1 worker with 1 CPU

Run	Run time (second)
1	84
2	84
3	84
4	78
5	78
6	84
7	78
8	84
9	84
10	84

Appendix B: Run time of 6 workers with 1 CPU each

Run	Run time (second)
1	66
2	66
3	66
4	66
5	72
6	66
7	66
8	66
9	66
10	66

Appendix C: Run time of 2 workers with 3 CPUs each

Run	Run time (second)
1	26
2	27
3	28
4	28
5	29
6	29
7	29
8	29
9	29
10	29